# Python Activity Record

**Courier Charge Optimization**

## 1. Problem Statement

Develop a Python program that calculates courier charges based on parcel weight, delivery zone, and delivery type. The program should display the costs of the allowed services, compare them with the user's budget, and recommend the lowest-cost service that fits the budget. It should display an error message when the supplied values are invalid.

## 2. Objectives

- To accept parcel weight, delivery zone, delivery type, and budget from the user.
- To validate the entered values.
- To calculate charges using service rates and zone fees.
- To filter services according to standard or express delivery selection.
- To recommend the cheapest eligible service within the given budget.

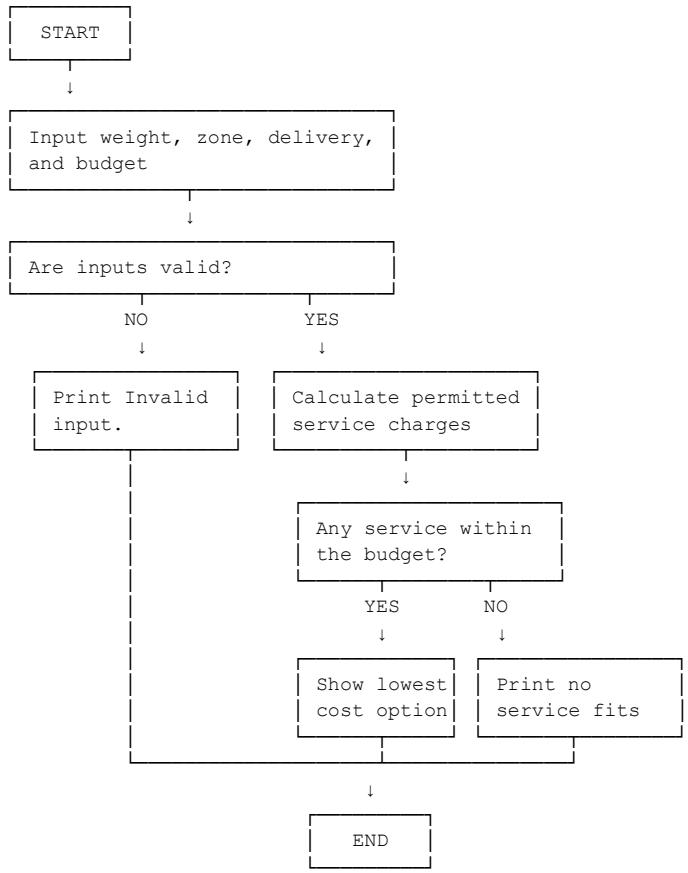
## 3. Assumptions

- Parcel weight is entered as a number greater than zero.
- The zone must be local, regional, or national.
- Delivery must be standard or express.
- Budget is entered as a number greater than or equal to zero.
- The rates and extra fees are fixed as defined in the source code.
- The code assumes numeric input for weight and budget; non-numeric input is not handled.

## 4. Algorithm

1. Start.
2. Input parcel weight, zone, delivery type, and budget.
3. Define zone fees and the available service rates.
4. Check whether the weight, budget, zone, or delivery type is invalid.
5. If invalid, print “Invalid input.” and stop.
6. Create an empty list to store services that fit the budget.
7. Check each service and skip services not allowed for the selected delivery type.
8. Calculate each allowed service cost using weight × rate + zone fee + extra fee.
9. Display each calculated cost and add it to the eligible list if it is within budget.
10. If eligible services exist, select the lowest-cost one and display it; otherwise, print “No service fits the budget.”
11. Stop.

## 5. Flowchart




## 6. Pseudocode

```
BEGIN
    INPUT weight, zone, delivery, budget
    zone_fee = {local: 0, regional: 50, national: 100}
    services = [(Standard, 40, 0), (Express, 65, 40), (Priority, 90, 80)]

    IF weight <= 0 OR budget < 0 OR zone invalid OR delivery invalid THEN
        PRINT "Invalid input."
    ELSE
        eligible = empty list
        FOR each service (name, rate, extra)
            Skip Standard when delivery is express
            Skip Express/Priority when delivery is standard
            cost = weight * rate + zone_fee[zone] + extra
            PRINT service cost
            IF cost <= budget THEN
                ADD (cost, name) to eligible
            END IF
        END FOR

        IF eligible is not empty THEN
            SELECT lowest-cost eligible service
            PRINT service name and cost
```

```
        ELSE
            PRINT "No service fits the budget."
        END IF
    END IF
END
```

## 7. Python Source Code

```
weight = float(input("Parcel weight (kg): "))

zone = input("Zone (local/regional/national): ").lower()

delivery = input("Delivery (standard/express): ").lower()

budget = float(input("Budget (Rs.): "))

zone_fee = {"local": 0, "regional": 50, "national": 100}

services = [("Standard", 40, 0), ("Express", 65, 40), ("Priority", 90, 80)]

if weight <= 0 or budget < 0 or zone not in zone_fee or delivery not in ("standard",
"express"):

    print("Invalid input.")

else:

    eligible = []

    for name, rate, extra in services:

        if delivery == "standard" and name != "Standard":

            continue

        if delivery == "express" and name == "Standard":

            continue

        cost = weight * rate + zone_fee[zone] + extra

        print(name, "cost: Rs.", round(cost, 2))

        if cost <= budget:

            eligible.append((cost, name))

    if eligible:

        cost, name = min(eligible)

        print("Best eligible service:", name, "Cost: Rs.", round(cost, 2))

    else:

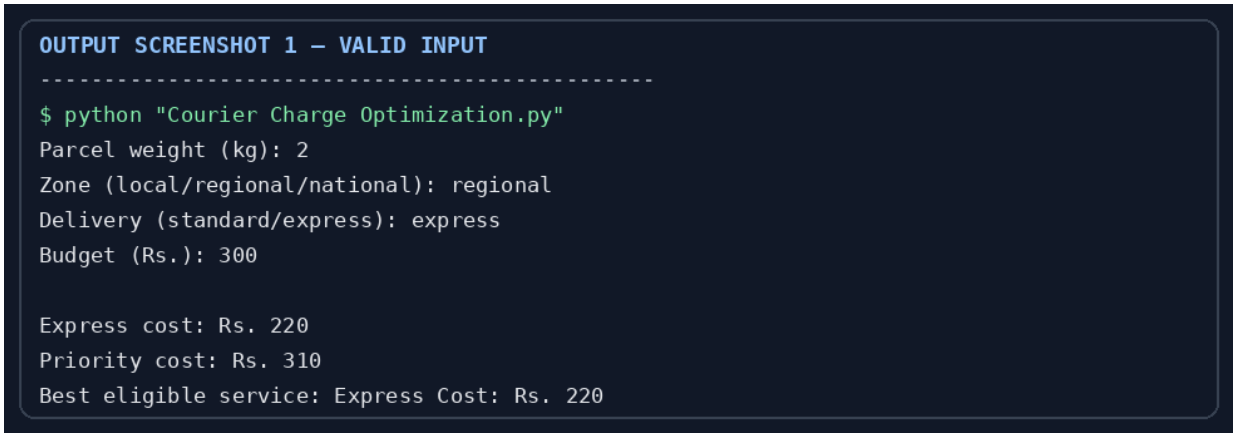
        print("No service fits the budget.")
```

## 8. Test Cases

| Test | Weight / Zone | Delivery / Budget | Expected result | Purpose |
|---|---|---|---|---|
| 1 | 2 kg / regional | express / Rs. 300 | Express Rs. 220; Priority Rs. 310; best eligible: Express Rs. 220 | Valid input and budget |
| 2 | 0 kg / local | standard / Rs. 100 | Invalid input. | Invalid weight |
| 3 | 5 kg / national | express / Rs. 100 | Express Rs. 465; Priority Rs. 630; No service fits the budget. | Budget too low |
| 4 | 1 kg / local | standard / Rs. 40 | Standard cost Rs. 40; best eligible: Standard Rs. 40 | Minimum valid weight and exact budget |
| 5 | 2 kg / unknown | standard / Rs. 200 | Invalid input. | Invalid zone |

## 9. Output Screenshots

Screenshot 1: Valid express delivery with regional zone.




Screenshot 2: Invalid parcel weight.

```
OUTPUT SCREENSHOT 2 — INVALID INPUT
------------------------------------------------
$ python "Courier Charge Optimization.py"
Parcel weight (kg): 0
Zone (local/regional/national): local
Delivery (standard/express): standard
Budget (Rs.): 100

Invalid input.
```

Screenshot 3: No service fits the budget.

```
OUTPUT SCREENSHOT 3 — BUDGET TOO LOW
------------------------------------------------
$ python "Courier Charge Optimization.py"
Parcel weight (kg): 5
Zone (local/regional/national): national
Delivery (standard/express): express
Budget (Rs.): 100

Express cost: Rs. 465
Priority cost: Rs. 630
No service fits the budget.
```

## 10. Conclusion

The Courier Charge Optimization program uses input validation, dictionaries, lists, loops, conditional statements, and minimum-value selection. It calculates charges for services allowed by the selected delivery type and recommends the lowest-cost service within the budget. If the inputs are invalid or no service is affordable, it displays an appropriate message.

Limitation: The supplied program expects numeric input for parcel weight and budget. Entering text instead of a number can raise a ValueError.